

Báo cáo Bài tập lớn Xác Suất Thống Kê

Phân Tích và Dự Đoán Giá Phát Hành GPU

Dựa Trên Các Thông Số Kỹ Thuật

Nhóm 16

Lớp L03

GVHD: Huỳnh Thái Duy Phương

TP. Hồ Chí Minh, 2026



Danh sách thành viên — Nhóm 16, Lớp L03

STT	Họ và tên	MSSV	Đóng góp
1	Trần Gia Tường	2313835	100%
2	Trần Nhật	2412490	100%
3	Trương Minh Triết	2413604	100%
4	Vũ Đại Dương	2410638	100%



Thay đổi mới so với file trước

Danh sách thay đổi

- Chạy Welch's ANOVA trên **log(Giá)** thay vì giá gốc.
- Kiểm tra giả định ANOVA bằng **Q-Q plot** theo từng hãng.
- Sắp xếp lại trình tự một số Slide.

- 1 Tổng quan đề tài
- 2 Cơ sở Lý thuyết
- 3 Thực thi Code và Xử lý dữ liệu
- 4 Kết quả Thu được
- 5 Kết luận

PHẦN 1

Tổng quan đề tài

Báo cáo Bài tập lớn Xác suất Thống kê — Nhóm 16 — L03

Hai câu hỏi nghiên cứu

Câu hỏi 1 — Hồi quy

Thông số nào **quyết định giá** GPU?

$$\log(\hat{Y}) = \beta_0 + \sum_{i=1}^k \beta_i X_i + \varepsilon$$

Mục tiêu: $R^2 \geq 70\%$, MAE nhỏ

Câu hỏi 2 — ANOVA

Giá GPU NVIDIA đắt hơn AMD có **ý nghĩa thống kê**?

$$H_0: \mu_{\text{NVIDIA}} = \mu_{\text{AMD}}$$

Welch's ANOVA (phương sai không đồng nhất)

Công cụ: R — dữ liệu thực tế, kiểm định nghiêm ngặt



All_GPUs.csv (Kaggle)

- **3.406** GPU: NVIDIA, AMD, Intel
- **34** thuộc tính gốc
- $\approx$ **84%** thiếu giá niêm yết

Vấn đề cốt lõi

34 thuộc tính $\xrightarrow{\text{lọc 3 tầng}}$ **7 đặc trưng** + 1 biến mục tiêu

Biến	Kiểu	Vai trò
Release_Price	Liên tục	Mục tiêu Y
Memory_Bandwidth	Liên tục	Độc lập
Max_Power	Liên tục	Độc lập
Core_Speed	Liên tục	Độc lập
Memory	Rời rạc	Độc lập
Release_Year	Số nguyên	Độc lập
Manufacturer	Phân loại	Độc lập
Memory_Type	Phân loại	Độc lập

PHẦN 2

Cơ sở Lý thuyết

Báo cáo Bài tập lớn Xác suất Thống kê — Nhóm 16 — L03

Đặc thù dữ liệu GPU cần xử lý

Bộ nhớ rời rạc

Bus/VRAM chỉ tồn tại theo chuẩn cứng (64/128/256 bit, 2/4/8 GB) — không thể nội suy tuyến tính

ATI $\equiv$ AMD

ATI bị AMD thôn tóm năm 2006; dataset có cả hai nhãn — cần hợp nhất

Intel: thiếu giá 100%

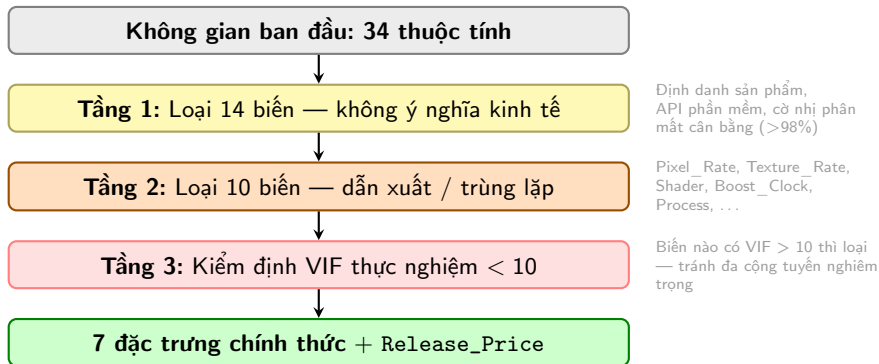
Toàn bộ GPU Intel không có Release_Price $\Rightarrow$ loại bỏ hoàn toàn

Định giá dị biệt

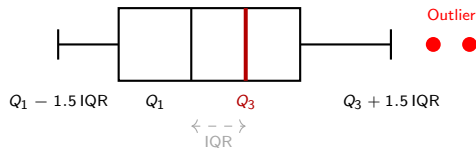
Workstation/Quadro, Titan limited, Dual-GPU, Hydro Copper $\Rightarrow$ lọc bỏ bằng regex



Thu hẹp đặc trưng: Lọc 3 tầng (34 → 7)



Phương pháp loại Outlier — IQR Tukey



Tại sao IQR (Tukey)?

- **Bền vững** với phân phối lệch — không bị kéo bởi flagship cực giá
- Không giả định phân phối chuẩn
- Dựa trên trung vị, không phụ thuộc trung bình

Nguyên tắc then chốt

IQR **phải tính trên dữ liệu gốc** trước điền khuyết.
Nếu điền trước $\Rightarrow$ IQR có giả tạo $\Rightarrow$ quan sát bình thường bị loại nhầm.

Chia tập dữ liệu & Chiến lược Điền khuyết

Chia Train/Test (90:10)

- Chia **trước** điền khuyết — tránh data leakage
- Train: $N = 401$ Test: $N = 45$
- `set.seed(2026252)` $\Rightarrow$ tái lập hoàn toàn

Nếu điền trước $\rightarrow$ chia sau: thống kê Test “rò” vào Train

Điền khuyết theo vai trò biến

Biến	Phương pháp	Tham số chỉ từ Train
Memory	Regex → Mode	
Số liên tục	Median	
Phân loại	Mode	
⇒ áp sang Test		

Lý do dùng Median (không phải Mean)

GPU flagship tạo đuôi phải mạnh — Median bền vững hơn Mean với ngoại lai

Tại sao dùng mô hình Log-Linear?

Vấn đề với mô hình Linear

- Giá GPU phân phối **lệch phải** mạnh (flagship tạo đuôi dài)
- Phương sai phần dư **không đồng nhất** (Heteroscedasticity)
- OLS giả định cộng gộp tuyệt đối — không phù hợp

Tại sao Log-Linear?

Thông số kỹ thuật tác động lên giá theo **tỷ lệ %**:

$$\log(Y) = \beta_0 + \sum \beta_i X_i + \varepsilon$$

β_i đo % thay đổi giá khi X_i tăng 1 đơn vị. Log nén đuôi, ổn định phương sai.

Các kiểm định thống kê sử dụng

Welch's ANOVA

Ý nghĩa: So sánh giá trị trung bình hai nhóm độc lập.

Mục tiêu: Kiểm định chênh lệch giá hãng NVIDIA-AMD khi phương sai không đồng nhất.

$H_0: \mu_{\text{NVIDIA}} = \mu_{\text{AMD}}$ (Welch hiệu chỉnh bậc tự do).

VIF — Đa cộng tuyến

Ý nghĩa: Đo lường mức độ tương quan giữa các biến độc lập.

Mục tiêu: Phát hiện sự trùng lặp thông tin gây sai lệch hệ số OLS.

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$
 (Ngưỡng < 10 , R_j^2 : hồi quy X_j lên các biến khác).

Breusch-Pagan

Ý nghĩa: Kiểm định phương sai sai số thay đổi (Heteroscedasticity).

Mục tiêu: Đảm bảo tính hiệu quả tuyến tính tốt nhất (BLUE) của OLS.

$H_0: \text{Var}(\varepsilon_i)$ không phụ thuộc X ($p > 0.05 \Rightarrow$ phương sai đồng nhất).

Shapiro-Wilk

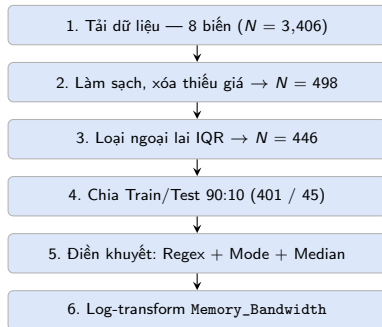
Ý nghĩa: Kiểm định giả định phần dư có phân phối chuẩn.

Mục tiêu: Đảm bảo tính hợp lệ cho các kiểm định hệ số t-test, F-test.

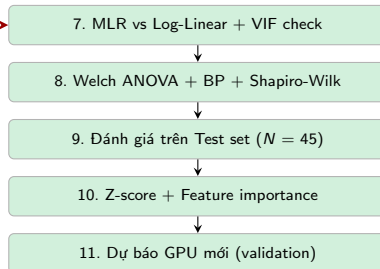
H_0 : phần dư $\sim \mathcal{N}(0, \sigma^2)$ (mẫu lớn kết hợp Q-Q Plot và CLT).

Pipeline phân tích: 11 bước

Tiền xử lý



Mô hình & Đánh giá



PHẦN 3

Thực thi Code và Xử lý dữ liệu

Báo cáo Bài tập lớn Xác suất Thống kê — Nhóm 16 — L03

Code: Tải dữ liệu và Làm sạch

```
kept_vars <- c("Name", "Release_Price", "Max_Power",  
  "Memory", "Memory_Bandwidth", "Core_Speed",  
  "Release_Date", "Manufacturer", "Memory_Type")  
raw_data <- read.csv("All_GPUs.csv",  
  stringsAsFactors=FALSE)  
selected_data <- raw_data %>% select(all_of(kept_vars))
```

```
clean_data <- selected_data %>%  
  mutate(across(where(is.character),  
    ~ifelse(str_trim(.) %in% c("", "-", "NA", "N/A"),  
      NA, str_trim(.)))) %>%  
  filter(Manufacturer != "Intel") %>%  
  mutate(Manufacturer =  
    str_replace(Manufacturer, "ATI", "AMD")) %>%  
  filter(!str_detect(Name, regex(  
    "Quadro|Crossfire|SLI|not.released|Titan",  
    ignore_case=TRUE))) %>%  
  mutate(Release_Year = as.integer(  
    format(as.Date(Release_Date, "%d-%b-%Y"), "%Y"))) %>%  
  select(-Release_Date) %>%  
  mutate(  
    Release_Price = as.numeric(  
      str_extract(Release_Price, "\\d+\\.?[\\d]*")),  
    # ... tương tự Max_Power, Memory, Memory_Bandwidth,  
    Core_Speed  
    Manufacturer = as.factor(Manufacturer),  
    Memory_Type = as.factor(Memory_Type)  
  ) %>% filter(!is.na(Release_Price))
```

Điểm kỹ thuật

- all_of() ném lỗi nếu tên cột sai — không silent drop
- across(where(is.character)) xử lý toàn bộ cột text trong 1 lần
- Regex lọc Quadro/Crossfire/SLI/Titan — loại định giá dị biệt
- as.factor() để R tự sinh dummy variables trong lm()
- str_extract() → as.numeric(): chuỗi không hợp lệ tự thành NA

Kết quả

3,406 $\xrightarrow{\text{lọc}}$ 498 quan sát có giá

Code: Outlier → Split → Điều chỉnh

```
# IQR Outlier Detection
calc_tukey <- function(vec) {
  v <- na.omit(vec)
  q1 <- quantile(v,0.25); q3 <- quantile(v,0.75)
  iqr <- q3 - q1
  list(lower=q1-1.5*iqr, upper=q3+1.5*iqr)
}
price_bnd <- calc_tukey(clean_data$Release_Price)
clean_data <- clean_data %>%
  filter(Release_Price <= price_bnd$upper
         | is.na(Release_Price))
# => Q1=150, Q3=349, ngưỡng=647.5
# => Loại 52, còn lại 446 obs
```

```
# Train/Test 90:10 - split TRUOC dien khuyet
set.seed(2026252)
train_indices <- sample(1:nrow(clean_data),
                        0.9*nrow(clean_data))
train_data <- clean_data[ train_indices,] # N=401
test_data <- clean_data[-train_indices,] # N=45
# Regex trich VRAM tu ten GPU (buoc 1)
train_data <- extract_vram(train_data)
test_data <- extract_vram(test_data)
# Dien khuyet - tham so chi tu train_data
impute_params <- list(
  Memory = get_mode(train_data$Memory),
  Core_Speed = median(train_data$Core_Speed, na.rm=T),
  Max_Power = median(train_data$Max_Power, na.rm=T),
  Memory_Bandwidth =
```

```
# Trich VRAM bang Regex tu ten GPU
extract_vram <- function(df) {
  df %>% mutate(
    mem_str = str_extract(Name, regex(
      "\\b(\\d+)\\s*(GB|G|MB)\\b",
      ignore_case=TRUE)),
    mem_val = as.numeric(
      str_extract(mem_str, "\\d+")),
    mem_val = ifelse(
      str_detect(mem_str,
        regex("GB|G\\b", ignore_case=TRUE)),
        mem_val*1024, mem_val),
    Memory = ifelse(is.na(Memory)
      & !is.na(mem_val), mem_val, Memory)
  ) %>% select(-mem_str, -mem_val)
}
```

Output IQR

Q1=150, Q3=349, IQR=199
Ngưỡng trên = 647.50
Ngoại lai: 52 obs bị loại
Còn lại: 446 obs

Code: Xây dựng & So sánh Mô hình

```
# Log-transform Memory_Bandwidth (phan phoi lech phai)
train_data <- train_data %>%
  mutate(log_Memory_Bandwidth = log(Memory_Bandwidth))
test_data <- test_data %>%
  mutate(log_Memory_Bandwidth = log(Memory_Bandwidth))

# Mo hinh 1: Linear OLS
mlr_model_linear <- lm(
  Release_Price ~ Max_Power + Memory +
  Memory_Bandwidth + Core_Speed +
  Manufacturer + Memory_Type + Release_Year,
  data = train_data)

# Mo hinh 2: Log-Linear (chinh thuc)
mlr_model_log <- lm(
  log(Release_Price) ~ Max_Power + Memory +
  log_Memory_Bandwidth + Core_Speed +
  Manufacturer + Memory_Type + Release_Year,
  data = train_data)

# VIF check ( $GVIF^{1/(2*Df)} < \sqrt{10} = 3.16$ )
car::vif(mlr_model_log)
```

VIF — Log-Linear model

Biến	VIF
log_Memory_BW	7.34
Release_Year	4.53
Max_Power	3.97
Core_Speed	3.64
Memory	3.17
Manufacturer	1.97
Memory_Type	4.06

Max VIF = 7.34 < 10

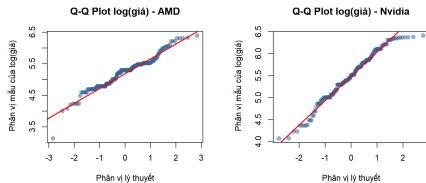
⇒ Không đa cộng tuyến

PHẦN 4

Kết quả Thu được

Báo cáo Bài tập lớn Xác suất Thống kê — Nhóm 16 — L03

Kết quả 1: NVIDIA cao hơn AMD 29,91%



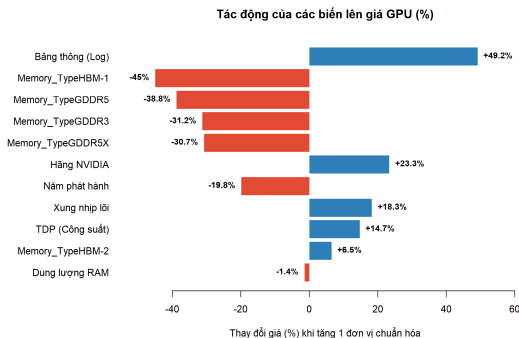
Welch's ANOVA trên $\log(\text{Giá})$: $F = 26,398$, $p = 4,556 \times 10^{-7}$

	NVIDIA	AMD	Quy đổi
n	182	219	–
Mean $\log(\text{Giá})$	5,4880	5,2263	$\Delta = 0,2617$
Geometric mean	\$241,77	\$186,11	+\$55,67
Tỷ lệ giá	NVIDIA/AMD = $e^{0,2617}$		+29,91%

Kết luận

$p \ll 0,001 \Rightarrow$ **Bác bỏ H_0** : log-giá trung bình NVIDIA cao hơn AMD có ý nghĩa thống kê.

Kết quả 2: Yếu tố nào quyết định giá?



Hệ số chuẩn hóa (Z-score)

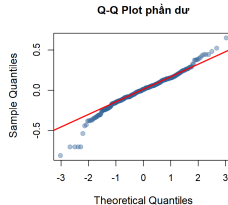
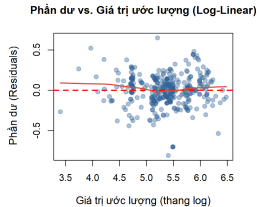
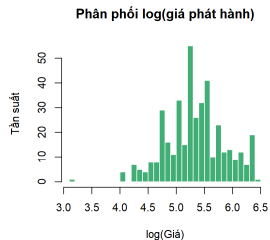
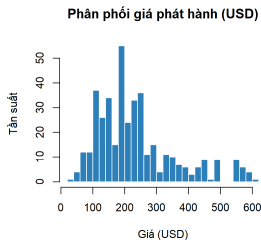
Adj. $R^2 = 86.62\%$

Biến	$\hat{\beta}$	
log(Bảng thông BN)	0.400	***
Năm phát hành	-0.221	***
NVIDIA premium	0.210	***
Xung nhịp lõi	0.168	***
Công suất TDP	0.137	***
Dung lượng RAM	-0.014	n.s.

Insight nổi bật

RAM size **không ý nghĩa** khi đã có bảng thông — tốc độ truyền dữ liệu quan trọng hơn dung lượng

Kết quả 3: Phân phối giá & Chẩn đoán mô hình



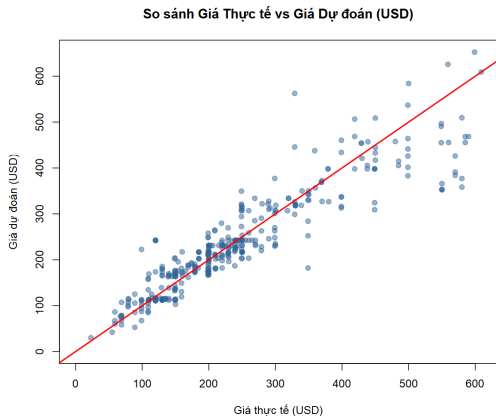
Residuals vs Fitted (trái) & Q-Q plot (phải)

Phân phối gốc (lệch phải) → sau log-transform (chuẩn)

Kiểm định giả định

VIF max	7.34	✓
Breusch-Pagan	$p = 0.196$	✓
Shapiro-Wilk	$W = 0.964$	≈✓

Kết quả 4: Hiệu năng dự đoán



So sánh mô hình (Test set)

Mô hình	RMSE	MAE	R^2
Linear	\$62.18	\$45.76	0.745
Log-Linear	\$51.92	\$33.50	0.823

Validation in-the-wild (3 GPU mới)

GPU	Linear	Log-Lin
GTX 1660	+14.2%	+6.8%
RX 590	+9.5%	-1.4%
RTX 3060	+25.5%	-4.4%

PHẦN 5

Kết luận

Báo cáo Bài tập lớn Xác suất Thống kê — Nhóm 16 — L03

Kết luận — Đánh giá kết quả nghiên cứu

CQ1 — Thông số nào quyết định giá?

Log-Linear ($R^2 = 86,99\%$, $MAE = \$33,50$):

- **Băng thông** ($\beta = 0,3999$) mạnh nhất
- Xung nhịp lõi, công suất Max Power tiếp theo
- **Dung lượng RAM không ý nghĩa** khi đã có tốc độ và loại RAM

CQ2 — NVIDIA đắt hơn AMD?

Welch's ANOVA trên $\log(\text{Giá})$ ($F = 26,398$, $p < 0,001$):

- Geometric mean: NVIDIA cao hơn **29,91%**
- Cùng cấu hình: NVIDIA đắt hơn **23,3%** ($e^{0,2097} - 1$) — phí CUDA

Kết quả vượt kỳ vọng

- Giải thích **86,99%** biến thiên giá
- Vượt kiểm định BP: $p = 0,1963$ ✓
- Dự báo in-the-wild: sai số chỉ 1%–7%

Năm phát hành âm ($\beta = -0,2208$)

Định lượng định luật giảm giá công nghệ theo thời gian một cách chính xác.



Hạn chế & Đề xuất

- 1 **Shapiro-Wilk** $p < 0,05$ do $N = 401$ đủ lớn — dùng CLT + Q-Q plot để đánh giá trực quan
- 2 **Selection Bias** ($> 80\%$ thiếu giá) — bổ sung giá lịch sử từ Internet
- 3 Chưa có **interaction terms** (VD: `max_power:manufacturer`)

Ý nghĩa thực tiễn

- 1 **Phát hiện GPU định giá sai** (sai số 1%–6%) — overpriced hay undervalued
- 2 **Định lượng “phí thương hiệu”**: $\approx 23,3\%$ ngân sách chỉ cho logo NVIDIA + CUDA
- 3 **Tối ưu ngân sách**: ưu tiên băng thông cao, công nghệ bộ nhớ (GDDR6, HBM) hơn dung lượng RAM lớn
- 4 **Xu hướng lỗi thời**: $\beta_{\text{year}} = -0,2208$ định lượng mức giảm giá theo năm

Cảm ơn thầy và các bạn

đã lắng nghe!